The Effects of Distance, Experience, and Communication Structure on Requirements Awareness in Two Distributed Industrial Software Projects

Irwin Kwan, Daniela Damian, and Sabrina Marczak
Department of Computer Science
University of Victoria
3800 Finnerty Road, Victoria, BC, Canada
{irwink, danielad, smarczak}@cs.uvic.ca

Abstract

In global software development, communication is difficult due to distance between sites. How effectively do team members distributed among multiple geographical locations become aware of changes and clarifications to requirements? In a case study of two different global software development projects, we observed how requirement analysts, developers, and testers maintain awareness of changes in the project. To gather data, we attended local and remote meetings, and conducted interviews of project team members. Based on our experience with these projects, we discuss the following awareness factors in software development: distance, experience of team members, and communication structure. We present the effects on awareness, and provide some lessons learned for global software development projects. We expect these lessons learned can be used by projects with similar settings.

1. Introduction

In software development, a software developer should keep aware of events that are occurring in a software development project. *Awareness*, in the context of software development, is whether a software developer working in a project has knowledge of events, such as changes to a requirement suggested by a customer, that occur in the project. The communication that informs a developer about an event is an awareness notification. This awareness becomes especially important in global software development, where distance can have an effect on the quality of communication among project team members [2, 10].

It is believed that informal communication is a strong contributor to awareness within a project [3]. Developers have been observed to spend a significant amount of time engaging in informal communication [11] and in group activities [9]. Lack of awareness is costly: A leader who does not maintain his knowledge of the application's design may be unable to manage the team effectively, and can lose contact with the customer [12]. A developer must be aware of the latest project developments and must remain synchronized with the information available to the rest of the team or problems with design, quality, and cost may occur.

This paper reports observations made during a case study regarding awareness in software development. We conducted a field study of two different globally-distributed industrial software projects in the Brazilian development centre of an American company. One project does distributed development divided among Brazil and United States (US) development offices, with its business clients in the US. The other project does co-located development in Brazil, with its business clients in the US.

The paper is organized as follows. We describe the context of the study, including the company and the projects, in Section 2. We describe the data collected in Section 3, and our observations in Section 4. We explain factors that may have affected awareness in Section 5 based on our observations and provide some lessons learned that may improve awareness in an organization in Section 6. We conclude the paper in Section 7.

2. Case Study Context

We conducted an observational study of two projects in the Brazilian subsidiary of an American company, which we will call ORG for confidentiality reasons. ORG has offices all over the world, including development centres in the US, Brazil, and India. ORG assembles and ships its products world-wide, and has an extensive I/T department to support its internal processes. We observed two projects in the Brazil office, selected according to availability and easiness to reach distributed members. Both projects involved communication and collaboration with remote clients in the

US. We describe the projects below; the project names have been changed to maintain anonymity.

Shipping System The Shipping System project (SHIP) updates and maintains an internal software product used by ORG to support its shipping process. The product is approximately seven years old, and is a critical component of the company's business. The teams are located in the US and in Brazil.

We were able to observe the last week of a maintenance release, and the first 6 weeks of an enhancement release. At the end of the observation period, the enhancement release had approximately 10 weeks in its schedule before deployment.

Support Applications The Support Applications project (APP) enhances and maintains a group of internal software products used in ORG by product management and sales. There are over 100 applications within this group, though about twenty are considered critical to the operation of the company. The applications are mature, and are undergoing regular enhancements and maintenance. The project contains a development group in Brazil, with the business partners in the United States.

We observed 6 weeks of a maintenance and enhancements release. At the end of the observation period, the release had 6 weeks remaining in its schedule before completion.

2.1. Project Team Organization

We examined the similarities and differences between two projects in an attempt to identify factors which may have had an effect on awareness in distributed development projects. We discuss the organization of the projects and provide a brief outline of their processes below.

Both projects build software that supports internal processes within the company. The requirements for the software come from internal clients, which we call *business partners* (BPs). BPs are employees of ORG, and may negotiate with external clients of ORG, may manage product portfolios, and may be users of the end product.

2.1.1 Shipping System: Experienced Team with a Decentralized Communication Structure

SHIP's teams are a development team distributed geographically across two sites, and a test team located in Brazil. The project manager, as well as the BPs, are in the US. There are five developers in the US, including the development lead with seven years experience, and the senior developer with five years of experience, allocated full-time to SHIP. There are two developers in Brazil allocated full-time to SHIP, including the Brazil development leader (Brazil dev lead) with four years experience, and two developers in Brazil allocated partially to SHIP. In addition, four contractors in a different building in Brazil are allocated in varying amounts to SHIP, though SHIP is their most important project. There are four testers fully allocated to SHIP, located in Brazil. There are two environment coordinators (ECs), one in Brazil, and one in US, who have to manage the development and test environments. The developers and testers coordinate closely with these ECs. The team members are working together for three years in average, and they had run many projects on Shipping System portfolio.

The BPs communicate primarily with the project manager, lead developer, and senior developer in US, as well as the Brazil dev lead. Developers and testers are not encouraged to contact the business partners directly unless it is through a team leader.

The team members in SHIP use face-to-face interaction, instant messenger, phone, and E-mail extensively; the latter three are used with remote team members. The Brazil dev lead often meets with the test lead and the EC face to face. The culture of the shipping system project encourages communication among team members, even across geographical boundaries. A team member is encouraged to contact any other team member for support. Because of this open communication among the team members, we consider SHIP as having a *decentralized communication structure*.

2.1.2 Support Applications: Inexperienced Team with a Centralized Communication Structure

APP is made up of 2 project managers (PMs), 5 business analysts, 7 testers, and 25 developers divided into 4 development teams in Brazil. One of these developers is the development lead coordinator (dev lead coordinator). In total, there are 39 people in the group. APP was formerly run by I/T employees in the US, but the company began an initiative to migrate application evolution to Brazil. Many of the applications were missing documentation, and many of the employees who had originally developed the applications had been transferred to other groups, or had left the company. Consequently, every team member in APP is new to the product, with the most senior person having four months of experience in the project.

In this structure, no cross-team communication is supposed to occur without notification to the leaders of each team. The team leaders and managers enforce the policy that all cross-team E-mail communication must be CCed to the appropriate team leaders (Figure 1). The purpose of the CC is to ensure that the team leader can intervene in the discussion to provide advice or feedback. Because the organization is designed to control the flow of information

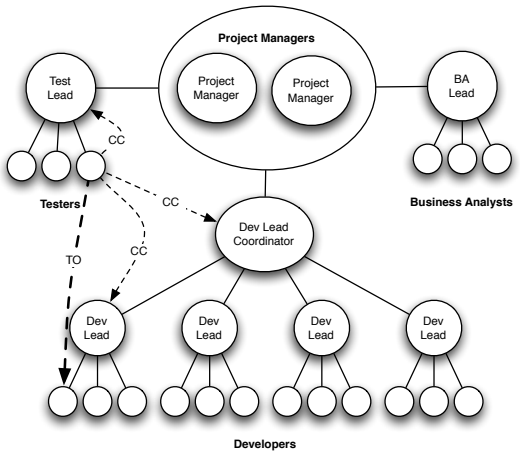


Figure 1. Organizational Structure of APP, with an example of a tester communicating with a developer

through team leads, we consider APP as having a *centralized communication structure*.

3. Data Collection and Analysis

We observed the software developers involved in each project in local meetings and remote conference calls. We also conducted semi-structured and free interviews during a period of two months. The semi-structured interviews scripts were prepared after a short period of observations, and the free interviews were conducted according events were taking place and calling the researchers attention.

We conducted 14 interviews with the SHIP developers. We interviewed 2 US developers, the test lead's US mentor, and 5 different Brazilian team members. We held multiple interviews with the development leader (4) and the test leader (2) to keep updated with project events, and to receive clarifications on observations.

We conducted 13 recorded interviews with the APP contributors in Brazil. We contacted 4 developers, 2 project managers, 1 tester, and 5 business analysts. We conducted two interviews with the BA lead.

We were able to gather project-related E-mail messages from every member of the SHIP team members in Brazil except 3. We were able to gather E-mail from only five APP team members.

Through analysis of observations notes and interview transcriptions we identified the awareness issues presented in exhibits on the next section.

4. Observations of Awareness in Industrial Practice in Distributed Development

In this section, we discuss exhibits which highlight awareness issues we observed within SHIP. These exhibits do not stress all situations observed. No awareness issues within APP became apparent to us. This feeling was corroborated with team members during interviews. It does not exclude the chances the issues were hidden, but it means they did not impact the team. We discuss what the awareness issue was, how it was resolved, and the potential effects on the project had the issue not been resolved in a timely fashion.

Exhibit 1: Domain knowledge not shared

A contractor in SHIP did not receive a document containing domain knowledge related to one of his requirements, and consequently lost an afternoon of work. The requirement was to redesign a shipping label used by a client to meet localization requirements. Because he did not receive all of the information to code requirements, he designed prototype labels to send to the leaders for feedback. The Brazil dev lead had a document from the client that contained clarifications to a number of requirements, but he had forgotten to forward it to the developer when the contractor was assigned the new label requirement.

The document, which describes the client's updated label standard, had been sent to every development lead by a BP in 2005. The Brazil dev lead received a new E-mail message on the BP list, which mentioned the label standard, which prompted him to look up the document from past E-mail archives and discuss it with the US development lead and the US senior developer.

Just before a weekly team meeting, the Brazil dev lead was discussing the prototypes that had been provided with the US senior developer when he recalled the document describing the label standard. They both discovered that the contractor did not receive the updated label standard. The contractor found out about this document in the local team meeting between all team members in Brazil.

Exhibit 2: Requirements clarifications late

The test team of SHIP did not receive requirement clarifications from the US project manager on time, despite the fact that the development team received them. The test lead in Brazil recalled a situation when she sent a list of requirements questions from her team to the project manager. She had also CCed to a senior developer in the United States. When the project manager received clarifications, he informed the developers, but forgot to inform the testers.

The senior developer in the US who received the original copy of the message realized that the test team had not

received these clarifications, and forwarded them in E-mail to the test leader, closing the awareness gap.

Although this situation was solved, a delay between sending the clarifications to the development team and sending the clarifications to the test team can mean that the test team works on outdated requirements which may lead to confusion especially when the development team and the test team synchronize.

Exhibit 3: Deadline not communicated

A meeting discussing the deadline for the final day of the planning phase did not involve the test team. The test team of SHIP was not informed about the exit date for the planning phase that was being discussed among the project manager and some developers. However, an experienced test lead from the US who was not allocated to the project, but was acting as a mentor for the current test leader in Brazil, was present in a conference call when she noticed that the project manager was mentioning dates that seemed suspicious to her. She spoke with the current test lead about the situation, and then confronted the project manager to ensure that he knew he was forgetting the test team in his planning.

The awareness issue was resolved quickly by the mentor and prevented any damage to the project. However, had the test team not been made aware of the planning phase deadline, coordination in the project would have been affected. They may not have finished their estimations, their task assignments, or their requirements questions before development started.

5. Factors Affecting Awareness

We have identified some factors that may influence awareness in each project. We have observed (1) the effect of a distributed development, (2) the effect of experienced team members, and (3) the effect of communication structure.

5.1. Effect of Distributed Development on Awareness

Numerous sources have shown that distance between team members has a significant impact on communication [1, 8, 4]. SHIP was affected by awareness issues because of the distance between its two development teams. In Exhibit 2, coordination creates a large delay in communication: The test team leader must contact the project manager for requirements clarification over a large distance, and the project manager must respond. Already, the response is delayed, which causes problems with the test team's work. However, the issue is compounded because distance also reduces the project manager's awareness of the test team. In Exhibit 2 and 3, we see that the project manager does not immediately contact the test team, and therefore causes them to wait longer than would otherwise be necessary. Fortunately, the presence of experienced members on the SHIP team, as well as open communication among these members, helped to mitigate the damage that awareness problems may have caused.

APP did not have the same delays because the developers were all colocated. This may be a reason why we did not notice any awareness issues within APP. Only the clients were remote, but BAs contacted the customer in the US regularly and were able to promptly answer most of the developers' questions.

5.2. Effect of Team Member Experience on Awareness

It is well-known that experienced team members provide great benefits to software engineering projects. One benefit we have observed is their contribution to identifying and resolving awareness gaps.

In both systems, experienced members take the role of coordinators. In SHIP, the experienced team members worked alongside each other team member, but were kept up-to-date frequently using both formal and informal communication, even across geographically-distributed sites. We have observed that, in every awareness exhibit, the gap in awareness was bridged by an experienced member who realised that there was a knowledge gap, and took actions to resolve the gap. For instance, in Exhibit 1, the Brazilian lead, with the assistance of the US senior developer, identified that the contractor did not have the label standard. Exhibit 2, the senior developer from the US forwarded information to the test team when she received a reply from the project manager regarding the test team's questions. In Exhibit 3, the test leader's mentor, from the US, identified that there was a gap in awareness between the project manager, and the test team. We see in these cases that an experienced member had an intuitive sense of where awareness gaps existed, and took measures to fill those gaps.

In APP, experienced team members, serving as team leaders, were hubs of communication, and were made aware of every message sent across teams that were related to the requirements or the design. The team leaders were copied on each message so that they could intervene, thus avoiding awareness problems.

5.3. Effect of Communication Structure on Awareness

The way that team members coordinate, based on the communication structure, has an effect on how aware each team member is. The difference between the SHIP and the

APP communication structures are vastly different, which may explain some of the awareness issues in SHIP.

The communication structure in SHIP is a decentralized structure. Although there are clearly-defined development leaders, the team members are not required to go through the development leaders when contacting each other, although there is extensive use of CCed E-mail. Because communication was not structured in this group, there were more communication paths (up to $\frac{(n-1)^2}{2}$ lines of communication [5]), and therefore, more possibilities for a gap in awareness. In Exhibit 1, the developer did not receive the label requirements document that was supposedly sent to each team member. Exhibit 2 and Exhibit 3 also highlight situations where a lack of structured communication caused the project manager to forget to contact the test leader. Despite the structure, the team was able to stay aware of its requirements, especially during the volatile planning phase, where requirements from the BPs came to the team very loosely-defined. We did not see these awareness problems in APP, where team leaders were able to intervene if they observed gaps in awareness.

In an interview with the development lead coordinator of APP, he stated that this structure is strong for new teams such as APP until they can gain more experience. He mentioned the volume of E-mail as a problem. The development lead coordinator also mentioned a vision for the team to move toward "more decentralized communication" as it gained more experience, and stated the advantages of decentralized communication as reduced communication on team leaders, and faster information exchange among team members. In this instance, we see a possible trade-off between how strict communication within a team is, and how aware team members in the team are.

6. Lessons Learned

Based on our experience as researchers with the case study, we have identified the following lessons in distributed software development. These lessons should be applicable to medium-sized distributed software projects with remote customers that have similar project settings.

6.1. Experienced Team Members Bridge Awareness Gaps

For most of the cases in which we observed an awareness gap, the gap was bridged by an experienced team member who had caught wind of the situation. In SHIP (the decentralized group), the senior developer from Brazil, and the senior developer from the US were able to intervene in order to provide information to team members.

Based on this observation, an organization should try to not only retain experienced team members in a team, but also make them as accessible as possible, especially if the experienced team member is remote.

6.2. Centralized Structures Keep New Teams Aware

We have seen that experienced team members contribute significantly in identifying and bridging awareness gaps. In APP, the team members were very new to the application domain, but yet, did not suffer from any awareness problems. In the team leaders were able to intervene if they observed gaps in awareness. Although we cannot claim that this was solely due to the use of a centralized structure, we believe that this may have had an influence in preventing awareness problems.

There are, however, some advantages to the decentralized structure. Although the decentralized structure experienced more awareness problems, we cannot claim that this structure is worse than a centralized structure, namely because the projects cannot be compared directly. We found that members in the decentralized team engaged in informal communication using E-mail and instant messenger frequently with members of the remote site—usually multiple times a day. This was in contrast to a study performed by Herbsleb and Grinter, which found that remote communication did not occur unless scheduled [8]. Other advantages to the decentralized structure include less dense communication and faster response times from colleagues, but the centralized structure may risk information overload [6]. Although there were awareness problems in SHIP, the gaps were closed quickly, perhaps due to swift, unobstructed communication even to remote sites.

A project may wish to strike a balance between a decentralized and a centralized structure. However, in an organization with team members who are new to the domain, the team members should communicate every message to their appropriate team leader to ensure that leaders are aware of what solutions are being discussed. Team leaders can easily contribute to the discussion and remain aware of what the team members are working on.

6.3. Frequent Meetings Improve Awareness among Local and Remote Teams

Groups in ORG often use meetings to synchronize information among project members. Every person interviewed in the study mentioned the importance of regular meetings to keep aware of project events. The majority of the groups met twice a week to synchronize information among each other. In SHIP, the group had one weekly meeting for everyone at Brazil, and an additional meeting which included the developers from the US on a different day of the week. A group from APP met as often as five days a week, but scaled

back to three days a week after the team leader received feedback that team members were not receiving enough new information during each meeting.

To maintain awareness and ensure that team members are up to date with the project's events, the local team members should meet face-to-face as a group, and remote team members should meet together despite the distance. ORG used conference calls for remote team meetings.

6.4. A Developer in a Distributed Team who has Multiple Tasks Reduces The Effect of Delays from Coordination

There is a significant delay when working in global software development [8, 7], but parallelizing work may help minimize the effect of this delay. When planning a project, especially in a global software project, a manager should consider assigning a number of stable requirements with an unstable requirement to the same developer. This will allow a developer to explore the unstable requirements and send appropriate questions to the remote team, or to the customer. While waiting, he can work on stable requirements and experience minimal downtime.

The developer in Exhibit 1 who did not receive the documentation from the lead developer told us that he was still productive even after he asked the business partners for requirements details because he had other tasks to do.

The effect of the increased delay in global software development may have been reduced because the developer could work on other tasks. Although some time was lost due to a lack of awareness about the requirement, the time may not have been significant because he was able to work on other, more stable requirements while waiting for more information.

7. Conclusion

Awareness ensures that each team member is up-to-date so he can do effective work. However, maintaining awareness in a distributed development environment is extremely difficult. From this case study of two distributed software projects, we observed the following factors as having an effect on awareness. (1) A project with distributed development sites reduces awareness. (2) Experienced team members bridge awareness gaps. (3) A centralized communication structure may prevent awareness problems, but at the cost of information overload [8].

There may be other factors that affect awareness in a software development environment that we have not observed. For example, requirements stability may be a factor. A project that stabilizes its requirements early reduces the need for communicating changes late in the project that may lead to awareness problems. Cultural issues may have an effect on communication and awareness. Our list of awareness factors is far from complete.

We present the following lessons in our study. (1) Experienced team members can bridge awareness gaps, so ensure they are accessible. (2) A centralized communication structure can help a new team keep aware. (3) Frequent meetings improve awareness and help detect awareness issues. (4) A developer in a distributed team with multiple tasks can be productive when there are delays.

This is by no means a comprehensive list of recommendations. This study is limited because the observations, though structured, are informal, and the sample size is small. These lessons may not be applicable to every distributed development project, but we believe that they are useful to those who wish to improve communication, coordination, and awareness in their projects.

References

[1] T. J. Allen. *Managing the Flow of Technology: Technology Transfer and the Dissemination of Technological Information Within the R&D Organization*. The MIT Press, January 1984.

[2] D. Damian. Stakeholders in global RE: Lessons learned from practice. *IEEE Software*, March 2007. To appear.

[3] D. E. Damian and D. Zowghi. The impact of stakeholders' geographical distribution on managing requirements in a multi-site organization. In *IEEE Joint International Conference on Requirements Engineering 2002, September 9–13*, pages 319–328, September 2002.

[4] K. Ehrlich and K. Chang. Leveraging expertise in global software teams: Going outside boundaries. In *International Conference on Global Software Engineering 2006, Florianopolis, Brazil.*, pages 149–158, 2006.

[5] J. Frederick Phillips Brooks. *The Mythical Man-Month*. Addison-Wesley, 1975.

[6] S. R. Fussell, R. E. Kraut, J. F. Lerch, W. L. Scherlis, M. M. Mcnally, and J. J. Cadiz. Coordination, overload and team performance: effects of team communication strategies. In *CSCW '98: Proceedings of the 1998 ACM conference on Computer supported cooperative work*, pages 275–284, New York, NY, USA, 1998. ACM Press.

[7] J. D. Herbsleb and R. E. Grinter. Architectures, coordination, and distance: Conway's law and beyond. *IEEE Software*, 16(5):63–70, 1999.

[8] J. D. Herbsleb and R. E. Grinter. Splitting the organization and integrating the code: Conway's law revisited. In *ICSE*, pages 85–95, 1999.

[9] J. D. Herbsleb, H. Klein, G. M. Olson, H. Brunner, J. S. Olson, and J. Harding. Object-oriented analysis and design in software project teams. *Human-Computer Interaction*, 10(2/3):249–293, 1995.

[10] J. D. Herbsleb, A. Mockus, T. A. Finholt, and R. E. Grinter. An empirical study of global software development: Distance and speed. In *ICSE '01: Proceedings of the 23rd International Conference on Software Engineering*, pages 81–90. IEEE Computer Society, 2001.

[11] D. E. Perry, N. A. Staudenmayer, and L. G. Votta. People, organizations, and process improvement. *IEEE Software*, 11(4):36–45, 1994.

[12] Y. Yamauchi, J. Whalen, N. Ikeya, and E. Vinkhuyzen. The problem of knowledge decoupling in software development projects. In *28th International Conference on Software Engineering, Shanghai, China*, pages 877–880, 2006.